

コンパイラ (2)

2026/10/6

構文解析

構文解析 (Parsing)

文字列のならばが規則通りにならんでいるかチェックすること。

字句解析 : 正規表現 (Regular Expression)

(トークンにまとめる) FA : 有限オートマトン

$$w \stackrel{?}{\in} L(FA)$$

構文解析 : 文脈自由文法 (Context Free Grammar)

(木構造を構成) PDA : プッシュダウンオートマトン

$$w \stackrel{?}{\in} L(PDA)$$

木構造を出力 : PDT プッシュダウントランスデューサー

構文解析の基礎

構文解析

字句解析の出力トークン列から文法に従った木構造を構成

文法 文脈自由文法 (CF 文法) (N, Σ, P, E)

非終端記号: $N = \{E\}$, 終端記号: $\Sigma = \{+, *, (,)\} \cup \text{ID}$

$$P = \left\{ \begin{array}{ll} E \rightarrow E + E, & E \rightarrow E * E, \\ E \rightarrow \text{ID}, & E \rightarrow (E) \end{array} \right\}$$

- (1) ID は式である。
- (2) x と y が式であれば、 $x + y, x * y, (x)$ は式である。
- (3) (1),(2) を有限回適用して得られる表現だけが式である。

BNF(Backus-Naur-Form)

$$\langle S \rangle ::= \langle S \rangle + \langle S \rangle \mid \langle S \rangle * \langle S \rangle \mid \text{ID} \mid (\langle S \rangle)$$

導出と還元

導出 : $w_1 \Rightarrow w_2$

$w_1, w_2 \in (N \cup \Sigma)^*$ $w_1 = wAw'$, $w_2 = wvw'$ に対して $A \rightarrow v$

導出関係

$v \xRightarrow{*} v' : \Rightarrow$ の 0 回以上の適用

$$v = v_0 \Rightarrow v_1 \Rightarrow \cdots \Rightarrow v_{n-1} \Rightarrow v_n = v'$$

還元 $w_1 \Rightarrow w_2$

w_2 は w_1 に還元される。

生成言語

$G = \langle N, \Sigma, P, S \rangle$ が生成する言語 :

$$L(G) = \{w \mid S \xRightarrow{*} w\} \subseteq \Sigma^*$$

$w \in L(G)$ のとき、文法に従った文、 $w \notin L(G)$ のとき、文法エラー

構文解析 = $w \in L(G)$ をチェックし、 $w \in L(G)$ のとき解析木を生成

最左導出、最右導出

$A \rightarrow \alpha \in P$ によって、 $\beta A \gamma$ を書き換える。

$$\beta A \gamma \Rightarrow \beta \alpha \gamma$$

最左導出： $\beta \in \Sigma^*$ $\Rightarrow_{lm}$

最右導出： $\gamma \in \Sigma^*$ $\Rightarrow_{rm}$

関数定義 $\Rightarrow_{lm}$ 型名 名前 '(' ')' 複文

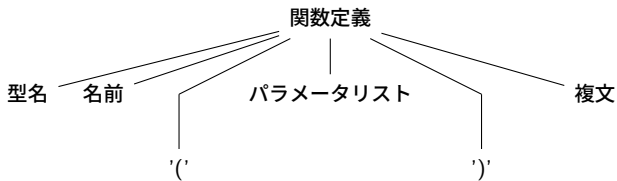
$\Rightarrow_{lm}$ INT 名前 '(' ')' 複文

$\Rightarrow_{lm}$ INT NAME '(' ')' 複文

$\Rightarrow_{lm}$ INT NAME '(' ')' COMPOUND

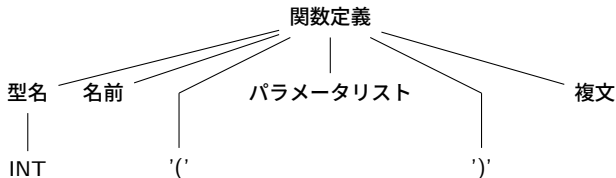
$$G' = \langle N, T, P, \text{関数定義} \rangle$$
$$N = \{ \text{関数定義, 型名, パラメータリスト, 名前, パラメータ, 複文} \}$$
$$T = \{ ' (' , ') ' , ' , ' , \text{VOID}, \text{INT}, \text{NAME}, \text{PARAM}, \text{COMPOUND} \}$$
$$P = \left\{ \begin{array}{ll} \text{関数定義} & \rightarrow \text{型名 名前 '(' パラメータリスト ')'} \text{ 複文} \\ \text{関数定義} & \rightarrow \text{型名 名前 '(' ')'} \text{ 複文} \\ \text{型名} & \rightarrow \text{VOID} \\ \text{型名} & \rightarrow \text{INT} \\ \text{型名} & \rightarrow \epsilon \\ \text{パラメータリスト} & \rightarrow \text{パラメータリスト ',' パラメータ} \\ \text{パラメータリスト} & \rightarrow \text{パラメータ} \\ \text{名前} & \rightarrow \text{NAME} \\ \text{パラメータ} & \rightarrow \text{PARAM} \\ \text{複文} & \rightarrow \text{COMPOUND} \end{array} \right.$$

解析木の導出



関数定義 $\Rightarrow$ 型名 名前 ' (' パラメータリスト ') ' 複文

解析木の導出



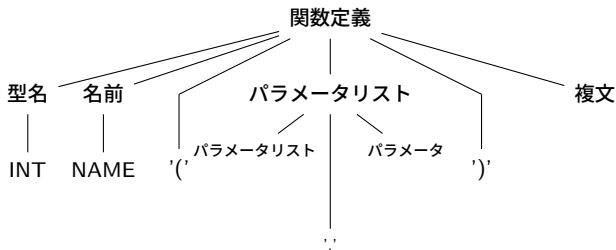
関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文

解析木の導出



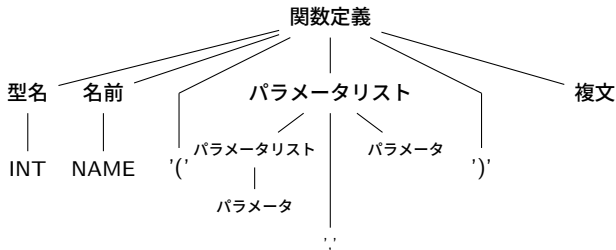
関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ')' 複文

解析木の導出



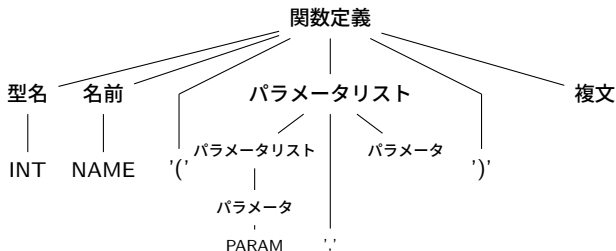
関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ',' パラメータ ')' 複文

解析木の導出



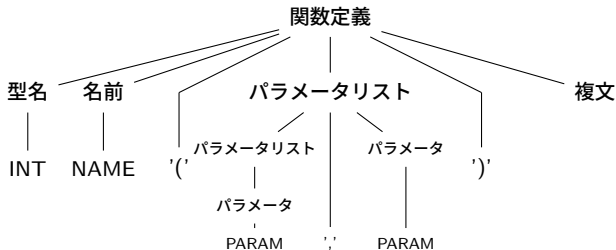
- 関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ',' パラメータ ')' 複文
⇒ INT NAME '(' パラメータ ',' パラメータ ')' 複文

解析木の導出



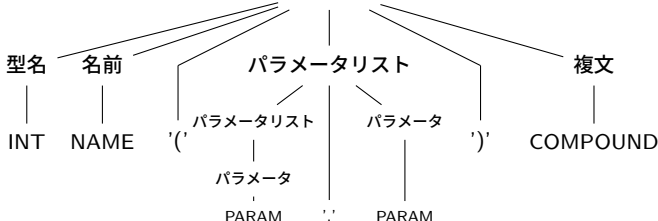
- 関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ',' パラメータ ')' 複文
⇒ INT NAME '(' パラメータ ',' パラメータ ')' 複文
⇒ INT NAME '(' PARAM ',' パラメータ ')' 複文

解析木の導出



- 関数定義 ⇒ 型名 名前 '(' パラメータリスト ')' 複文
⇒ INT 名前 '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ')' 複文
⇒ INT NAME '(' パラメータリスト ',' パラメータ ')' 複文
⇒ INT NAME '(' パラメータ ',' パラメータ ')' 複文
⇒ INT NAME '(' PARAM ',' パラメータ ')' 複文
⇒ INT NAME '(' PARAM ',' PARAM ')' 複文

関数定義



関数定義	⇒	型名 名前 '(' パラメータリスト ')' 複文
	⇒	INT 名前 '(' パラメータリスト ')' 複文
	⇒	INT NAME '(' パラメータリスト ')' 複文
	⇒	INT NAME '(' パラメータリスト ',' パラメータ ')' 複文
	⇒	INT NAME '(' パラメータ ',' パラメータ ')' 複文
	⇒	INT NAME '(' PARAM ',' パラメータ ')' 複文
	⇒	INT NAME '(' PARAM ',' PARAM ')' 複文
	⇒	INT NAME '(' PARAM ',' PARAM ')' COMPOUND

文法の曖昧性

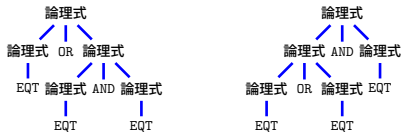
$G = \langle N, T, S, P \rangle$ ある $w \in L(G)$ に対して、 $S \Rightarrow w$ であって構成される解析木が唯一でないとき、文法が曖昧である。

$\Rightarrow$ 意味が正確に定まらないことがある。

教科書 G_1 : 曖昧な文法

$$G_1 = (\{ \text{論理式} \}, \{ \text{OR}, \text{AND}, \text{EQT} \}, P_1, \text{論理式})$$

$$P_1 = \{ \text{論理式} \rightarrow \text{論理式 OR 論理式} \mid \text{論理式 AND 論理式} \mid \text{EQT} \}$$



トークン EQT を左からそれぞれ $x == 0, y == 0, z == 0$ として、図 4.1 の解析木を、 $x = 0, y = 1, z = 1$ としてみると、左の解釈では、論理式は真であるが、右の解釈では偽。

無曖昧で等価な文法

G_2 は、 G_1 と等価な無曖昧な文法 ($L(G_1) = L(G_2)$)

解析木のサイズは大きくなる

$G_2 = (\{ \text{論理式}, \text{AND 式} \}, \{ \text{OR}, \text{AND}, \text{EQT} \}, P_2, \text{論理式})$

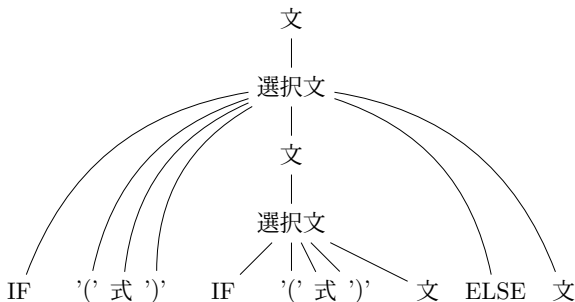
$P_2 = \left\{ \begin{array}{l} \text{論理式} \rightarrow \text{論理式 OR AND 式} \mid \text{AND 式} \\ \text{AND 式} \rightarrow \text{AND 式 AND EQT} \mid \text{EQT} \end{array} \right\}$



効率の面から、曖昧性を残して別のルールで一意に定めることもある。
－ や ÷ の左結合性 ($x - y - z$ は普通 $(x - y) - z$ と解釈)

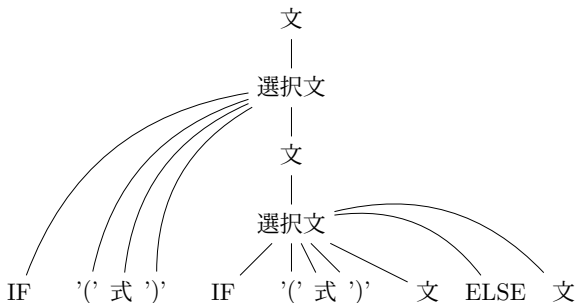
Dangling IF

文 $\rightarrow$ 選択文
選択文 $\rightarrow$ IF '(' 式 ')' 文
選択文 $\rightarrow$ IF '(' 式 ')' 文 ELSE 文



Dangling IF

文 $\rightarrow$ 選択文
選択文 $\rightarrow$ IF '(' 式 ')' 文
選択文 $\rightarrow$ IF '(' 式 ')' 文 ELSE 文



構文解析アルゴリズム

プリオーダー、インオーダー、ポストオーダー：木構造に対する
1次元表現

ソースコード → 木構造	構文解析
木構造 → オブジェクトコード	コード生成

長さ n の入力：CFG に対して $O(n^3)$ で構文解析可能
標準形の CFG に対して組み合わせをかぞえあげる
CFG によっては、もっと高速に構文解析が可能
文法規則の適用可能性をできるだけ早く判断

解析木のプリオーダー：最左導出
解析木のポストオーダー：最右導出の逆

First と Follow

どの生成規則を適用できるか、生成規則の集合からあらかじめ計算して手掛かりを得ておく。

定義 4.2 $First_1$ 集合

CF 文法 $G = \langle N, \Sigma, P, S \rangle$ が与えられたとき、 $\alpha \in (N \cup \Sigma)^*$ に対する $First_1(\alpha)$ を以下のように定義する。

$\alpha \Rightarrow^* \varepsilon$ のとき

$$First_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\} \cup \{\varepsilon\}$$

$\alpha \not\Rightarrow^* \varepsilon$ のとき

$$First_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\}$$

$First_1(\alpha)$: α を導出するときの最初のシンボル

$\varepsilon \in First_1(\alpha)$: α が ε に還元される

First₁ の計算

$PO_P: P$ の右辺の接尾語集合

$\gamma_1\gamma_2\cdots\gamma_n$ の接尾語: $\gamma_1\cdots\gamma_n, \gamma_2\cdots\gamma_n, \gamma_3\cdots\gamma_n, \dots, \gamma_n, \varepsilon$

アルゴリズム 4.1

ステップ 1: $\text{First}_1(\varepsilon) \leftarrow \{\varepsilon\}$

ステップ 2: $\alpha \in (PO_P \setminus \{\varepsilon\}) \cup N$ に対して

- (a) $\text{fst}(\alpha) \in \Sigma$ ならば, $\text{First}_1(\alpha) \leftarrow \{\text{fst}(\alpha)\}$
- (b) $\text{fst}(\alpha) \in N$ ならば, $\text{First}_1(\alpha) \leftarrow \emptyset$

ステップ 3: すべての $\text{First}_1(\alpha)$ ($\alpha \in PO_P \cup N$) に新たな要素が追加されなくなるまで以下を繰り返す.

- (a) $\text{fst}(\alpha) = A \in N$ である α に対して $\text{First}_1(\alpha)$ に以下の要素を追加する.
 $\varepsilon \notin \text{First}_1(A)$ のとき, $\text{First}_1(A)$
 $\varepsilon \in \text{First}_1(A)$ のとき, $\text{First}_1(A) \setminus \{\varepsilon\} \cup \text{First}_1(\text{fol}(\alpha))$
- (b) $A \rightarrow \beta \in P$ に対して $\text{First}_1(\beta)$ を $\text{First}_1(A)$ に追加する.

$\text{fst}(\alpha)$: 系列 α の先頭の記号

$\text{fol}(\alpha)$: 系列 α の先頭の記号を除いた残りの系列

Follow₁

$A \in N$ の Follow: $S \Rightarrow^* \beta A a \gamma$ となる $a \in \Sigma \cup \{\#\}$ をすべて求める。

下向き構文解析: $A \Rightarrow^* \varepsilon$ の場合の $A \rightarrow \alpha$ の適用

上向き構文解析: $B \rightarrow \alpha A$ となる規則の適用

定義 4.3 Follow₁

CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して, $A \in N$ に対する $\text{Follow}_1(A)$ は以下のように定義される。

$$\begin{aligned} \text{Follow}_1(A) = & \{a \in \Sigma \mid S \Rightarrow^* \beta A a \gamma, \beta, \gamma \in (N \cup \Sigma)^*\} \\ & \cup \{\# \mid S \Rightarrow^* \beta A, \beta \in (N \cup \Sigma)^*\} \end{aligned}$$

ここで, # は入力の終わりを示す記号である。

$Follow_1$ の計算

アルゴリズム 4.2

ステップ 1: $Follow_1(S) \leftarrow \{\#\}$

ステップ 2: $A \in N \setminus \{S\}$ に対して $Follow_1(A) \leftarrow \emptyset$

ステップ 3: $B \rightarrow \beta A \gamma \in P$ を満たす $B, A \in N, \beta, \gamma \in (N \cup \Sigma)^*$ に対してすべての $Follow_1(A) (A \in N)$ に新たな追加がなくなるまで追加する。

- (a) $\varepsilon \notin First_1(\gamma)$ のとき, $First_1(\gamma)$
- (b) $\varepsilon \in First_1(\gamma)$ のとき, $First_1(\gamma) \setminus \{\varepsilon\} \cup Follow_1(B)$

例

$$G = (\{X, Y, Z\}, \{a, c, d\}, P, Z)$$

$$P = \left\{ \begin{array}{ll} X \rightarrow a, & X \rightarrow Y, \\ Y \rightarrow c, & Y \rightarrow \varepsilon, \\ Z \rightarrow d, & Z \rightarrow XYZ \end{array} \right\}$$

$$P = \left\{ \begin{array}{ll} X \rightarrow a, & X \rightarrow Y, \\ Y \rightarrow c, & Y \rightarrow \varepsilon, \\ Z \rightarrow d, & Z \rightarrow XYZ \end{array} \right\}$$

	(1)	(2)	(3a) ¹	(3b) ¹	(3a) ²	(3b) ²	(3a) ³	(3b) ³	(3a) ⁴	(3b) ⁴
X	{ ε }	$\emptyset$	$\emptyset$	{a}	{a}	{a, c, ε }	{a, c, ε }	{a, c, ε }	{a, c, ε }	{a, c, ε }
Y		$\emptyset$	$\emptyset$	{c, ε }	{c, ε }	{c, ε }	{c, ε }	{c, ε }	{c, ε }	{c, ε }
Z		$\emptyset$	$\emptyset$	{d}	{d}	{d, a}	{d, a}	{d, a, c}	{d, a, c}	{d, a, c}
a		{a}	{a}	{a}	{a}	{a}	{a}	{a}	{a}	{a}
c		{c}	{c}	{c}	{c}	{c}	{c}	{c}	{c}	{c}
d		{d}	{d}	{d}	{d}	{d}	{d}	{d}	{d}	{d}
ε		{ ε }	{ ε }	{ ε }	{ ε }	{ ε }	{ ε }	{ ε }	{ ε }	{ ε }
XYZ		$\emptyset$	$\emptyset$	$\emptyset$	{a}	{a}	{a, c, d}	{a, c, d}	{a, c, d}	{a, c, d}
YZ		$\emptyset$	$\emptyset$	$\emptyset$	{c, d}	{c, d}	{a, c, d}	{a, c, d}	{a, c, d}	{a, c, d}

ここでは (3) を 3 回実行した時点で変化がなくなった. (4 回目は変化がないことを確認するための実行)

$$P = \left\{ \begin{array}{ll} X \rightarrow a, & X \rightarrow Y, \\ Y \rightarrow c, & Y \rightarrow \epsilon, \\ Z \rightarrow d, & Z \rightarrow XYZ \end{array} \right\}$$

	(1)	(2)	(3) ¹	(3) ²
X		$\emptyset$	$\{a, c, d\}$	$\{a, c, d\}$
Y		$\emptyset$	$\{a, c, d\}$	$\{a, c, d\}$
Z	$\{\#\}$	$\{\#\}$	$\{\#\}$	$\{\#\}$

これ以上増えないのは明らか

First/Follow の適用

同じ左辺を持つ複数の生成規則がある場合

$$1: A \rightarrow \alpha$$

$$2: A \rightarrow \beta$$

$\varepsilon \notin \text{First}(\alpha) \wedge \varepsilon \notin \text{First}(\beta)$ の場合

最左導出の先頭の文字が $\text{First}(\alpha)$ である場合、規則 1 を適用

最左導出の先頭の文字が $\text{First}(\beta)$ である場合、規則 2 を適用

$\varepsilon \in \text{First}(\alpha) \vee \varepsilon \in \text{First}(\beta)$ の場合

($\varepsilon \in \text{First}(\alpha)$ とすると) 最左導出の先頭の文字が

$\text{First}(\alpha) \setminus \{\varepsilon\} \cup \text{Follow}(A)$ の場合、規則 1 を適用

最左導出の先頭の文字が $\text{First}(\beta)$ である場合、規則 2 を適用

上記の方法で適用する規則が決まれば決定的に最左導出が可能

下降型構文解析

▶ 下降型構文解析アルゴリズム (再帰降下法)

$$\begin{aligned}r_1 : S &\rightarrow AB \\r_2 : A &\rightarrow aA \\r_3 : A &\rightarrow b \\r_4 : B &\rightarrow CDe \\r_5 : C &\rightarrow \varepsilon \\r_6 : C &\rightarrow c \\r_7 : D &\rightarrow \varepsilon\end{aligned}$$

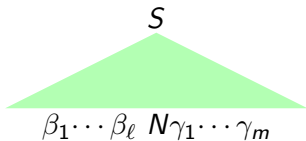
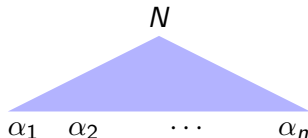
左辺の非終端記号を右辺の記号列で置き換えることで導出し、終端記号系列に置き換える。

$\Rightarrow_i$: 規則 r_i を適用して導出

$$\begin{aligned}S &\Rightarrow_1 \underline{A}B \Rightarrow_2 \underline{aA}B \Rightarrow_2 aa\underline{A}B \Rightarrow_3 aa\underline{b}B \\&\Rightarrow_4 aab\underline{CDe} \Rightarrow_6 aabc\underline{D}e \Rightarrow_7 aabce\end{aligned}$$

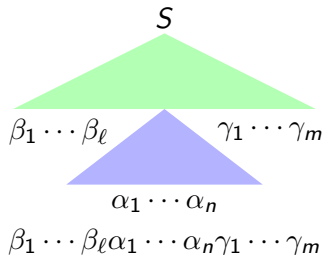
下向き構文解析の概念

$$N \rightarrow \alpha_1 \cdots \alpha_n$$

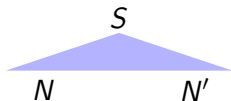


$$S \Rightarrow^* \beta_1 \cdots \beta_\ell N \gamma_1 \cdots \gamma_m$$

$\Rightarrow$



再帰降下法による導出



$$S \rightarrow NN'$$

$$N \rightarrow N''$$

$$N' \rightarrow M$$

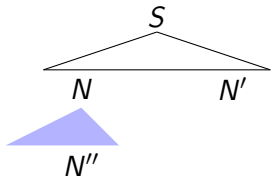
$$N'' \rightarrow a_1 \cdots a_k$$

$$M \rightarrow b_1 \cdots b_p$$

$$S \Rightarrow NN' \Rightarrow N''N' \Rightarrow a_1 \cdots a_k N' \Rightarrow a_1 \cdots a_k M \Rightarrow a_1 \cdots a_k b_1 \cdots b_p$$

終端記号の系列が導出されれば終了

再帰降下法による導出



$$S \rightarrow NN'$$

$$N \rightarrow N''$$

$$N' \rightarrow M$$

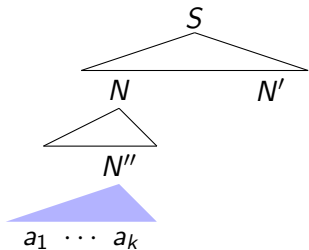
$$N'' \rightarrow a_1 \cdots a_k$$

$$M \rightarrow b_1 \cdots b_p$$

$$S \Rightarrow NN' \Rightarrow N''N' \Rightarrow a_1 \cdots a_k N' \Rightarrow a_1 \cdots a_k M \Rightarrow a_1 \cdots a_k b_1 \cdots b_p$$

終端記号の系列が導出されれば終了

再帰降下法による導出



$$S \rightarrow NN'$$

$$N \rightarrow N''$$

$$N' \rightarrow M$$

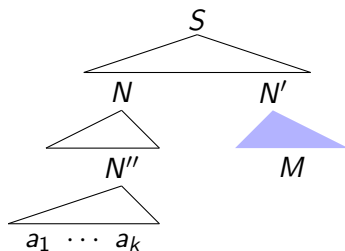
$$N'' \rightarrow a_1 \cdots a_k$$

$$M \rightarrow b_1 \cdots b_p$$

$$S \Rightarrow NN' \Rightarrow N''N' \Rightarrow a_1 \cdots a_k N' \Rightarrow a_1 \cdots a_k M \Rightarrow a_1 \cdots a_k b_1 \cdots b_p$$

終端記号の系列が導出されれば終了

再帰降下法による導出



$$S \rightarrow NN'$$

$$N \rightarrow N''$$

$$N' \rightarrow M$$

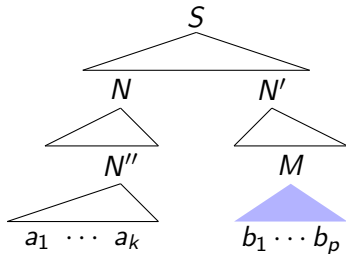
$$N'' \rightarrow a_1 \cdots a_k$$

$$M \rightarrow b_1 \cdots b_p$$

$$S \Rightarrow NN' \Rightarrow N''N' \Rightarrow a_1 \cdots a_k N' \Rightarrow a_1 \cdots a_k M \Rightarrow a_1 \cdots a_k b_1 \cdots b_p$$

終端記号の系列が導出されれば終了

再帰降下法による導出



$$S \rightarrow NN'$$

$$N \rightarrow N''$$

$$N' \rightarrow M$$

$$N'' \rightarrow a_1 \dots a_k$$

$$M \rightarrow b_1 \dots b_p$$

$$S \Rightarrow NN' \Rightarrow N''N' \Rightarrow a_1 \dots a_k N' \Rightarrow a_1 \dots a_k M \Rightarrow a_1 \dots a_k b_1 \dots b_p$$

終端記号の系列が導出されれば終了

再帰降下法の問題点

生成規則の左辺に同じ非終端記号をもつ規則が複数ある場合、どの規則を適用すべきか？

まちがった規則を適用すると解析木が生成できない

$$\{A \rightarrow a, A \rightarrow b\}$$

に対して、入力 b に対して、1 番目の規則を適用しても構文解析できない。後戻り (back track) して別の規則を適用すればよい

後戻りは計算機として一般に重い処理

→ あらかじめどの規則を適用できるか知りたい

生成規則を予め解析して *First/Follow* を得る

入力の文字を先読みする (Look-ahead)

先の例は、次の文字が b であることを知っていたら、2 番目の規則を後戻りなく適用可能

左括りだし (Left factoring)

生成規則に以下のような規則が含まれるとする。

$$A \rightarrow \alpha\beta_1$$

$$A \rightarrow \alpha\beta_2$$

を以下のように書き換えることを**左括り出し**

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1$$

$$A' \rightarrow \beta_2$$

導出は変わらない

降下型構文解析に不要なバックトラックを避ける。

そのままでは下向き構文解析不能であっても、等価で解析可能な文法に変換できる

左再帰性

- ▶ 右辺の非終端記号の呼出の順序はなんでもよい。生成規則の右辺に左辺と同じ記号が表れる。(再帰性)
最初から処理していく場合に、
右辺の最初に同じ非終端記号あらわれたら？(左再帰性)

パラメータリスト → パラメータリスト パラメータ
パラメータリスト → パラメータ

ずっと最初の規則を適用することがおこりうる。

パラメータリスト $\neq^* \epsilon$ であるとする、

$\text{First}(\text{パラメータリスト パラメータ}) = \text{First}(\text{パラメータリスト})$

2つの規則は First で見分けがつかない

左再帰性除去

左再帰性は以下のように除去可能

パラメータリスト $\rightarrow$ パラメータ パラメータリスト'

パラメータリスト' $\rightarrow$ ',' パラメータ パラメータリスト'

パラメータリスト' $\rightarrow \varepsilon$

一般に

$$A \rightarrow A\alpha_1, A \rightarrow A\alpha_2, \dots, A \rightarrow A\alpha_m$$
$$A \rightarrow \beta_1, A \rightarrow \beta_2, \dots, A \rightarrow \beta_n$$

は

$$A \rightarrow \beta_1 A', A \rightarrow \beta_2 A', \dots, A \rightarrow \beta_n A'$$
$$A' \rightarrow \alpha_1 A', A' \rightarrow \alpha_2 A', \dots, A' \rightarrow \alpha_m A', A' \rightarrow \varepsilon$$

という等価な文法に変換可能 $\Rightarrow$ 左再帰性除去

左再帰性除去

左再帰性は以下のように除去可能

$$\begin{aligned}\text{パラメータリスト} &\rightarrow \text{パラメータ } \text{パラメータリスト}' \\ \text{パラメータリスト}' &\rightarrow ', ' \text{パラメータ } \text{パラメータリスト}' \\ \text{パラメータリスト}' &\rightarrow \varepsilon\end{aligned}$$

一般に

$$\begin{aligned}A &\rightarrow A\alpha_1, A \rightarrow A\alpha_2, \dots, A \rightarrow A\alpha_m \\ A &\rightarrow \beta_1, A \rightarrow \beta_2, \dots, A \rightarrow \beta_n\end{aligned}$$

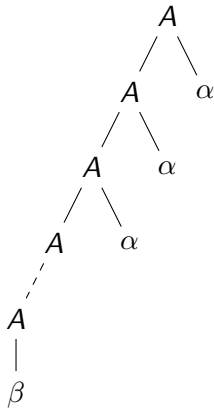
は

$$\begin{aligned}A &\rightarrow \beta_1 A', A \rightarrow \beta_2 A', \dots, A \rightarrow \beta_n A' \\ A' &\rightarrow \alpha_1 A', A' \rightarrow \alpha_2 A', \dots, A' \rightarrow \alpha_m A', A' \rightarrow \varepsilon\end{aligned}$$

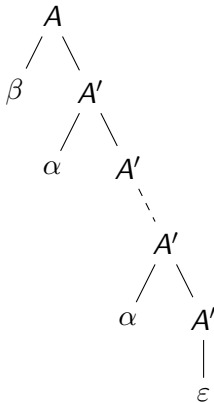
という等価な文法に変換可能 $\Rightarrow$ **左再帰性除去**

左再帰性

$$\left\{ \begin{array}{l} A \rightarrow A\alpha \\ A \rightarrow \beta \end{array} \right\}$$



$$\left\{ \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \\ A' \rightarrow \varepsilon \end{array} \right\}$$



下向き構文解析表

最左導出における A を含む文形式は $\beta A \gamma$ ($\beta \in \Sigma^*$)

A を左辺にもつ生成規則： $A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$

入力系列 $w = \beta\beta'$ に対して $S \Rightarrow^* w$ となるための $\text{fst}(\beta'\#)$ と $\text{First}_1(\alpha_i)$ との対応表

縦に N 、横に $\Sigma \cup \{\#\}$ を書く。

- ▶ $a \in \text{First}_1(\alpha_i)$ ならば、 A の行、 a の列に $A \rightarrow \alpha_i$ を追加
- ▶ $\varepsilon \in \text{First}_1(\alpha_i)$ ならば、 A の行、 $a \in \text{Follow}_1(A)$ の列に $A \rightarrow \alpha_i$ を追加

First

$$\left\{ \begin{array}{ll} X \rightarrow a, & X \rightarrow Y, \\ Y \rightarrow \varepsilon, & Y \rightarrow c, \\ Z \rightarrow d, & Z \rightarrow XYZ \end{array} \right\}$$

X	$\{a, c, \varepsilon\}$
Y	$\{c, \varepsilon\}$
Z	$\{a, c, d\}$
XYZ	$\{a, c, d\}$
YZ	$\{a, c, d\}$

Follow

X	$\{a, c, d\}$
Y	$\{a, c, d\}$
Z	$\{\#\}$

	a	b	c	#
X	$X \rightarrow a$ $X \rightarrow Y$	$X \rightarrow Y$	$X \rightarrow Y$	
Y	$Y \rightarrow \varepsilon$	$Y \rightarrow \varepsilon$ $Y \rightarrow c$	$Y \rightarrow \varepsilon$	
Z	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$	$Z \rightarrow d$ $Z \rightarrow XYZ$	

LL(1) 構文解析

LL(1) 文法

1 文字先読み ($First_1$ 集合) によって、下向き構文解析表を作成したとき、下向き構文解析表のエントリーに高々1つの生成規則しかない場合、LL(1) 文法とよぶ。

LL(1) 構文解析

LL(1) 文法 G では入力 w に対して $w \in L(G)$ であるとき、かつ、そのときに限り、1つの先読みトークンによって決定される単一の生成規則を用いて最左導出が可能である。

先読み文字で選ばれた生成規則で導出できないときは他の生成規則を使っても導出は構成不可能である。

LL: Left to right scan and left most derivation

ELL(1) 文法

拡張した生成規則に対しても全く同じ方法で下向き構文解析表が構成できる。

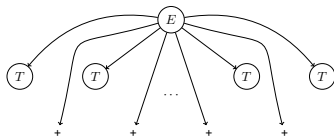
ELL(1) 文法

拡張した生成規則によって定義される拡張 CF 文法の下向き構文解析表を作成したとき、表のエントリの生成規則が高々 1 つである場合、ELL(1) 構文解析可能であり、その拡張 CF 文法は ELL(1) 文法である。

ELL(1) 文法における解析木の構成

$E \rightarrow E + T, E \rightarrow T$ から左再帰性を除去して拡張 CF 文法の規則
とすると $E \rightarrow T(+T)^*$

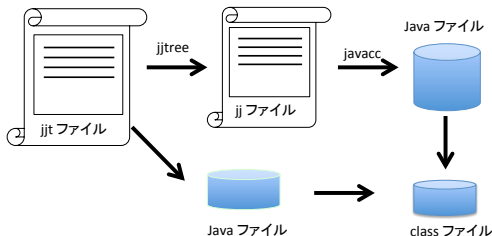
このため、そのままでは以下のような構文木を表す可能性がある。



この木構造は加算の意味を表しているとは言えない。もとの木と同じ意味を持つことを保証するためには、 $+$ が 2 項演算であることを示すための補助的情報を追加する必要がある。

jjtree による再帰降下型パーザの生成

javacc に含まれる jjtree をつかうと文脈自由文法から再帰降下構文解析プログラムを生成できる。



- (1) 字句定義と文脈自由文法を記述したファイル (jjt ファイル) を jjtree で変換して、いくつかの java ファイルと javacc ファイル (jj ファイル) を生成する。
- (2) javacc のファイルを javacc で java に変換して、全部を class ファイルに変換して完成

例 (四則演算式)

以下の (拡張)BNF 定義で定義される文法の構文解析を考える。

$$\begin{aligned}\langle \text{AddExpr} \rangle &::= \langle \text{MulExpr} \rangle ((+|-) \langle \text{MulExpr} \rangle)* \\ \langle \text{MulExpr} \rangle &::= \langle \text{PrimExpr} \rangle ((*|/) \langle \text{PrimExpr} \rangle)* \\ \langle \text{PrimExpr} \rangle &::= '(' \langle \text{AddExpr} \rangle ')' \mid \text{'Literal'}\end{aligned}$$

'Literal' は数のトークンであるとする。

$\langle \text{AddExpr} \rangle$ と $\langle \text{MulExpr} \rangle$ にはそれぞれ1つしか生成規則はなく、
 $\langle \text{PrimExpr} \rangle$ については、 $\text{First}_1('(' \langle \text{AddExpr} \rangle ')') = \{ '(' \}$ 、
 $\text{First}_1(\text{'Literal'})$ は互いに素であることから $ELL(1)$ 文法である。

参考・引用：五月女 健治：

Java CC: コンパイラ・コンパイラ for Java

テクノプレス (2003)[絶版]

```
void AddExpr() #void :
{
{
    MulExpr()
    (
        <ADDOP> MulExpr() #AddNode(2)
        | <SUBOP> MulExpr() #SubNode(2)
    )*
}
```

```
void MulExpr() #void :
{
{
    PrimExpr()
    (
        <MULOP> PrimExpr() #MulNode(2)
        | <DIVOP> PrimExpr() #DivNode(2)
    )*
}
```

```
void PrimExpr() #void :
{
{
    Literal()
    | <LP> AddExpr() <RP>
}
```

```
void Literal() :
{ Token t; }
{
    t=<LITERAL>
    { jjtThis.litValue
      = Integer.parseInt(t.image); }
}
```

式の計算

jjtree は下降型構文解析のプログラムのみを生成する。
生成された javacc ファイルは主に字句解析を行う。

生成された AddNode, SubNode, MulNode, DivNode において子供の値を計算する calculate メソッドを実装すればよい。

jjtree で生成した java プログラムに追加する。

講義の website を参照